

Détection de panneaux de signalisation routière par méthodes classiques

Groupe C — SY32 (P2026)
Université de technologie de Compiègne

Juin 2026

Résumé

Nous présentons un programme qui retrouve les panneaux de signalisation dans une image et entoure chacun d'une boîte accompagnée d'un score de confiance, en n'utilisant que des méthodes classiques (l'apprentissage profond est interdit). Le fil conducteur de notre travail tient en une phrase : *on ne peut améliorer que ce qu'on sait mesurer*. Nous avons donc d'abord construit de quoi évaluer honnêtement le détecteur (des métriques et un jeu de validation), puis amélioré la chaîne étape par étape, en gardant chaque idée seulement si les chiffres la confirmaient. La détection repose sur une fenêtre glissante multi-échelle, un descripteur **HOG + couleur + texture** et un **Random Forest**, complétés par du *hard negative mining*, un pré-filtrage couleur et une suppression des non-maxima. Sur notre validation, l'*Average Precision* est passée de 0,095 à 0,293, surtout grâce à un meilleur découpage des exemples et au *hard negative mining*. La première soumission sur le serveur d'évaluation a aussi révélé un défaut majeur — une précision très basse due à un excès de fausses détections — que nous analysons et corrigeons en fin de rapport.

1 Introduction

Le but est de détecter les panneaux dans des photos de rue : pour chaque image, le programme doit produire une ou plusieurs boîtes englobantes, chacune avec un score indiquant à quel point il « croit » qu'il s'agit d'un panneau. La règle imposée est de n'employer que des méthodes classiques de vision et d'apprentissage (`scikit-learn`, `scikit-image`, `OpenCV`), sans réseau de neurones profond.

Une difficulté supplémentaire est annoncée : le jeu de test, fourni seulement en fin de projet et *sans* annotations, contient des images *fish-eye* (objectif très grand-angle qui courbe l'image). Il faut donc penser à cette variation dès le départ.

Tout au long du rapport, nous suivons le même fil : (1) se donner les moyens de mesurer, (2) mettre en place une chaîne simple qui fonctionne, (3) l'améliorer en vérifiant à chaque fois l'effet réel sur les performances.

2 Le jeu de données et sa préparation

Les annotations. Chaque image contenant des panneaux est accompagnée d'un fichier `.csv`. Chaque ligne `y` décrit une boîte sous la forme `ymin`, `xmin`, `hauteur`, `largeur`, `difficulté`, où la difficulté vaut 0 (panneau facile) ou 1 (panneau difficile). L'ordre est donc « ligne puis colonne » (`y`, `x`), puis (`hauteur`, `largeur`) : ce n'est pas l'ordre habituel (`x`, `y`, `largeur`, `hauteur`), un point auquel il faut faire attention dans le code.

Ce que contient le jeu. Le jeu annoté (entraînement + validation) compte **341 images** et **662 boîtes**. La figure 1 en résume trois caractéristiques utiles pour la suite :

- *Tailles* : la plupart des panneaux sont petits à moyens (plus petit côté médian ≈ 79 px), avec une longue traîne de petits panneaux lointains. On relève **14 panneaux** sous le seuil de 16 px que l'énoncé demande d'écarter.
- *Difficulté* : **458** panneaux « faciles » pour **204** « difficiles ».
- *Forme* : le rapport largeur/hauteur est centré sur 1 (panneaux quasi carrés), ce qui justifie des imagettes carrées 64×64 .

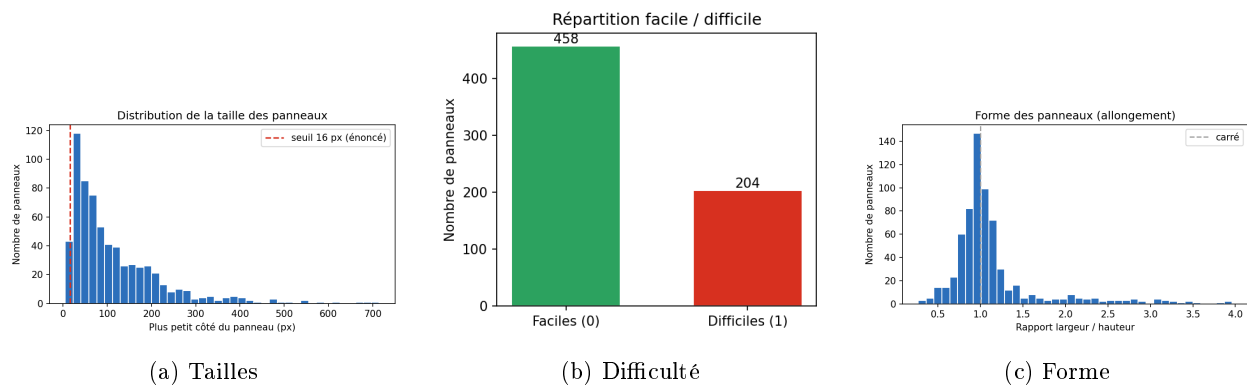


FIGURE 1 – Statistiques du jeu de données annoté (341 images, 662 boîtes).

La correction de la base. En relisant les annotations, nous avons repéré et traité quelques anomalies : une boîte de taille nulle (fichier 0309), des boîtes dont le plus petit côté fait moins de 16 px (que l'énoncé dit de ne pas annoter), et un déséquilibre du champ difficulté. Par sécurité, notre code d'évaluation ignore de lui-même les boîtes de surface nulle.

Comment on a évalué. Le jeu de test n'ayant pas d'annotations, on ne peut pas y calculer nos scores nous-mêmes. Nous avons donc mis de côté une partie des images d'entraînement pour en faire un jeu de *validation*. Le partage se fait *par image* et non par fenêtre : toutes les fenêtres d'une image de validation sont donc totalement inconnues du modèle ; sinon, des bouts d'une même image se retrouveraient des deux côtés et le score serait trompeusement bon.

Notre travail s'est déroulé en deux phases, avec deux jeux de validation différents (ils ne sont donc pas comparables entre eux) :

- **Phase 1** : ancien partage, **273 images** d'entraînement / **68** de validation. C'est là qu'on a obtenu les gros gains (découpage, hard negatives).
- **Phase 2** : après nettoyage et re-téléchargement de la base, nouveau partage **329** / **12**. Validation plus petite (donc AP plus bruitée, à lire avec prudence), utilisée pour régler le filtre couleur et la texture.

Remarque pour la livraison. Pour le rendu final, les images de validation sont réintégrées à l'entraînement afin que le modèle livré apprenne sur *toutes* les données. Les chiffres présentés ici restent ceux de la validation, c'est là qu'on a jugé chaque idée.

3 Comment fonctionne notre détecteur

Notre programme est organisé en plusieurs fichiers Python (un par rôle) : chargement des données, calcul des descripteurs, entraînement, détection, métriques, et des scripts qui orchestrent le tout. Voici la chaîne complète, dans l'ordre.

3.1 Préparer les exemples

Le classifieur a besoin d'exemples « panneau » et « pas panneau », tous de même taille.

- **Exemples positifs** : pour chaque boîte annotée, on découpe le panneau dans l'image. Comme les panneaux ne sont pas tous carrés, on agrandit le côté le plus court en prenant du *vrai contexte* autour (pas des bandes noires), puis on redimensionne le tout en 64×64 **pixels**.
- **Exemples négatifs** : on prend au hasard des morceaux d'images *sans* panneau (10 morceaux par image), eux aussi ramenés en 64×64 .

3.2 Décrire une imagerie : forme + couleur + texture

Une imagerie 64×64 n'est pas donnée telle quelle au classifieur : on la résume par un *descripteur*, c'est-à-dire une liste de nombres qui capture l'essentiel.

- **HOG** (histogramme de gradients orientés, 1764 valeurs) : il mesure, par petites zones, dans quelles directions les contours sont marqués. Il décrit donc la *forme* (cercle, triangle, etc.).
- **Histogramme de couleur HSV** (84 valeurs) : il résume les couleurs présentes. Utile car les panneaux ont des couleurs typiques (rouge, bleu, jaune).
- **Histogramme de texture LBP** (*Local Binary Pattern*, 10 valeurs) : il décrit la régularité locale de la surface, pour aider à séparer un panneau lisse d'un fond texturé (feuillage, brique).

Les trois descripteurs sont mis bout à bout en un seul vecteur de **1858 nombres**. C'est ce vecteur qui représente l'imagette.

3.3 Entraîner le classifieur, puis le corriger (hard negative mining)

Le classifieur est un **Random Forest** (« forêt aléatoire ») : un ensemble de 100 arbres de décision qui votent ; la proportion de votes « panneau » donne un score entre 0 et 1.

L'entraînement se fait en deux temps :

1. **Premier entraînement** sur les positifs et les négatifs décrits ci-dessus.
2. **Hard negative mining** : on fait passer le modèle sur beaucoup de morceaux d'images *sans* panneau, à plusieurs tailles. Tous ceux qu'il prend à tort pour des panneaux sont ses « erreurs » ; on les rajoute aux négatifs et on réentraîne. Le modèle apprend ainsi à ne plus se faire piéger par ces fonds trompeurs. Cette étape est rapide car on évalue tous les morceaux d'un coup.

3.4 Détecter sur une image entière

Pour une nouvelle image, on cherche les panneaux ainsi :

1. **Pré-filtrage couleur** : on construit un masque des pixels « couleur panneau » (rouge, bleu ou jaune saturés) et, grâce à une *image intégrale*, on ne lancera le classifieur que sur les fenêtres assez colorées. Cela écarte d'emblée le ciel, la route ou les façades grises, ce qui accélère et réduit les fausses détections. Un garde-fou désactive le filtre pour les images très sombres (scènes de nuit), où les couleurs sont peu saturées, afin de ne pas rater ces panneaux.
2. **Pyramide d'images** : on regarde l'image à sa taille normale, puis de plus en plus réduite (chaque niveau est 1,25 fois plus petit). Comme la fenêtre de recherche garde une taille fixe (64×64), réduire l'image revient à chercher des panneaux de plus en plus grands.
3. **Fenêtre glissante** : à chaque niveau, une fenêtre 64×64 se déplace par pas de 16 pixels. Chaque position retenue donne une imagette, qu'on décrit (HOG + couleur + texture) et qu'on classe.
4. **Score et seuil** : pour l'*évaluation*, on garde un seuil bas (0,3) afin de tracer toute la courbe de performance ; pour la *soumission*, on relève ce seuil pour ne garder que les détections sûres (voir section 6).
5. **Remise à l'échelle** : les positions trouvées sur une image réduite sont ramenées aux coordonnées de l'image d'origine.

3.5 Nettoyer les boîtes en double (NMS)

La fenêtre glissante détecte souvent le même panneau plusieurs fois. La **suppression des non-maxima** (NMS) garde, parmi des boîtes qui se chevauchent, celle au meilleur score et supprime les autres. Deux boîtes sont jugées « en double » si elles se recouvrent beaucoup (IoU élevé) *ou* si l'une est largement contenue dans l'autre. Ce second critère corrige un défaut de la NMS classique : une petite boîte à l'intérieur d'une grande a un IoU très faible et passerait donc à travers (détaillé en section 5).

3.6 Sauvegarder le modèle et produire le fichier de test

Entraîner prend du temps ; on ne veut pas recommencer à chaque détection. Une fois le modèle prêt, on l'enregistre dans `modele_panneaux.joblib` (bibliothèque `joblib`). Le script de prédiction recharge ce fichier au lieu de tout refaire ; une option `retrain` force un nouvel entraînement quand le code a changé. Ce fichier de modèle fait partie des livrables demandés.

Le script de test parcourt ensuite toutes les images du dossier `test/`, applique la détection puis la NMS, et écrit un seul fichier `detection.csv` au format exigé : Num img, ymin, xmin, hauteur, largeur, score, une ligne par détection.

3.7 Récapitulatif des paramètres

TABLE 1 – Principaux paramètres de la chaîne.

Paramètre	Valeur
Taille des imagerie	64×64 px
Pas de la fenêtre glissante	16 px
Réduction entre niveaux de pyramide	$\times 1,25$
Seuil de score (évaluation)	0,3
Seuil de score (soumission)	0,6
Seuil d'IoU pour la NMS	0,3
Seuil d'IoU vérité/détection	0,5
Pré-filtre couleur (fraction min.)	0,05
Négatifs aléatoires par image	10
Tailles des morceaux (hard neg.)	64, 96, 128, 200, 300 px
Random Forest	100 arbres
Descripteur HOG	9 orientations, cellules 8×8 , blocs 2×2
Descripteur couleur	histogramme HSV (36+32+16)
Descripteur texture	histogramme LBP uniform (P=8, R=1)
Taille du vecteur final	1858

4 Comment on mesure la performance

Pour décider si une idée est bonne, il faut un chiffre fiable. Nous avons codé nos propres métriques (fichier `metrics.py`).

IoU (recouvrement de deux boîtes). On compare une boîte trouvée à une vraie boîte en regardant la surface commune divisée par la surface totale :

$$\text{IoU}(A, B) = \frac{\text{aire}(A \cap B)}{\text{aire}(A \cup B)}. \quad (1)$$

Une détection est juste (un *vrai positif*) si son IoU avec une vraie boîte non encore prise est $\geq 0,5$; sinon c'est une fausse détection. Une même vraie boîte ne peut être comptée qu'une fois.

Précision et rappel. La *précision* dit « parmi ce que j'ai détecté, quelle part est correcte » ; le *rappel* dit « parmi les vrais panneaux, combien j'ai retrouvés » :

$$\text{Précision} = \frac{VP}{VP + FP}, \quad \text{Rappel} = \frac{VP}{VP + FN}. \quad (2)$$

AP (aire sous la courbe précision/rappel). En classant toutes les détections du score le plus haut au plus bas, précision et rappel évoluent : on obtient une courbe (figure 2), et l'*Average Precision* en est l'aire. C'est notre note principale. Nous rapportons aussi le **F1** (compromis entre précision et rappel) ainsi que la précision et le rappel au point où le F1 est le meilleur.

Deux seuils différents (à ne pas confondre). Le *seuil de score* décide quelles détections on garde selon la confiance du modèle ; il ne change rien à l'entraînement. Le *seuil d'IoU* mesure un recouvrement géométrique : 0,3 dans la NMS (repérer les doublons) et 0,5 pour décider si une détection tombe bien sur un vrai panneau.

5 Notre progression, étape par étape

5.1 Le point de départ : un piège

La toute première chaîne obtenait **98,96 % de bonnes réponses** quand on lui donnait des imagerie déjà découpées... mais seulement **AP = 0,095** en vraie détection. L'explication : bien classer une imagerie déjà centrée est facile, mais sur une image entière on teste des milliers de fenêtres, et même 1 % d'erreur fait apparaître des milliers de fausses boîtes. *Leçon : le score sur imagerie est trompeur ; seule la détection sur image entière compte.* C'est ce constat qui a justifié toute la suite.

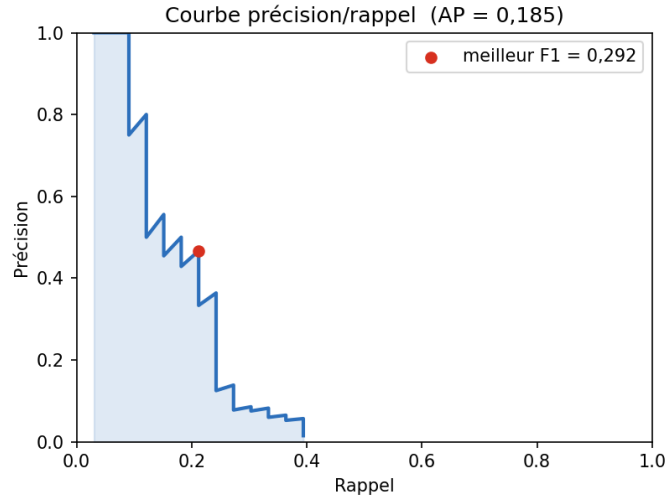


FIGURE 2 – Courbe précision/rappel du détecteur final sur la validation. Le point rouge marque le meilleur F1. L'aire sous la courbe est l'AP.

5.2 Mieux découper les exemples positifs

Au départ, les panneaux non carrés étaient complétés par des *bandes noires* pour devenir carrés. Le HOG apprenait alors ces bords noirs, qui n'existent jamais dans une vraie fenêtre. En les remplaçant par du vrai contexte de l'image, l'AP monte de **0,095 à 0,173** (+82 %). Premier vrai gain.

5.3 Apprendre de ses erreurs : hard negative mining

Le détecteur voyait des panneaux partout. En lui réinjectant ses fausses détections comme contre-exemples, l'AP grimpe de **0,173 à 0,293** (+69 %), et le nombre de détections *baisse* de 19341 à 11039 : il se trompe beaucoup moins. La figure 3 et le tableau 2 résument cette progression.

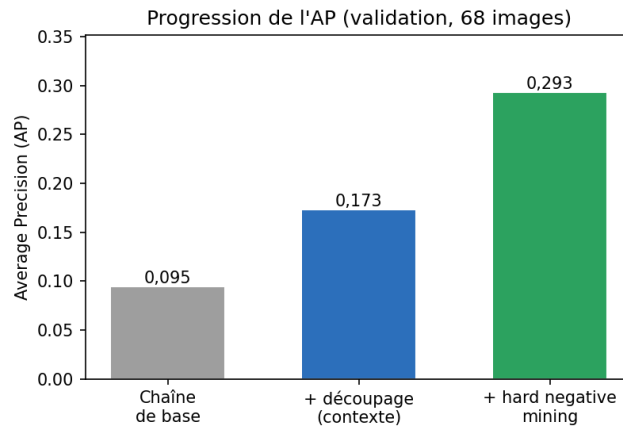


FIGURE 3 – Évolution de l'AP au fil des deux grandes améliorations (phase 1, 68 images).

TABLE 2 – Progression sur la validation (phase 1 : 68 images, 147 panneaux).

Étape	AP	F_1	Détections
Chaîne de base	0,095	0,224	8 511
+ meilleur découpage (contexte)	0,173	0,272	19 341
+ hard negative mining	0,293	0,387	11 039

5.4 Couleur et texture : pré-filtre + LBP

Sur le jeu nettoyé (phase 2, 12 images), nous avons ajouté le *pré-filtrage couleur* puis le descripteur de *texture LBP*. Le pré-filtre couleur ne change quasiment pas l'AP mais réduit fortement le nombre de fausses détections et le temps de calcul (jusqu'à $\approx 4\times$ plus rapide en version stricte). Le LBP, lui, fait remonter l'AP et le rappel ensemble (tableau 3, figure 4) : il aide réellement à distinguer un panneau d'un fond texturé. *Réserve honnête* : ces écarts sont mesurés sur seulement 12 images, donc bruités ; ils indiquent une tendance, pas une preuve définitive.

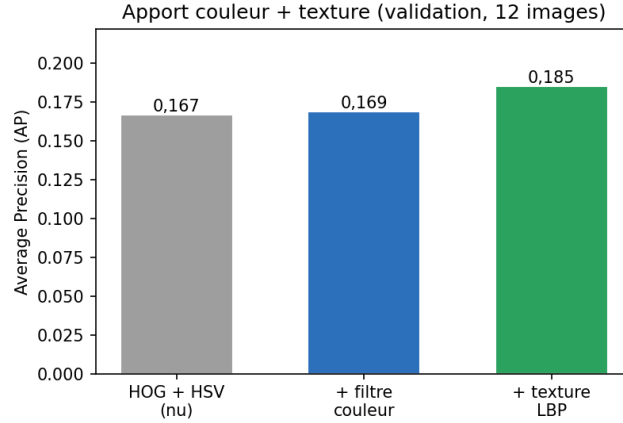


FIGURE 4 – Apport du filtre couleur puis du LBP (phase 2, validation 12 images).

TABLE 3 – Ablation des descripteurs (phase 2 : 12 images, 33 panneaux).

Configuration	AP	F_1	Rappel	Détections
HOG + HSV (nu)	0,167	0,273	0,182	1 218
+ filtre couleur	0,169	0,273	0,182	1 042
+ texture LBP	0,185	0,292	0,212	877

5.5 Nettoyer les boîtes imbriquées (NMS)

En regardant les détections, on voyait plusieurs boîtes empilées sur un même panneau. La NMS classique ne les supprimait pas car une petite boîte dans une grande a un IoU minuscule (par exemple 64×64 dans 305×305 donne un IoU d'environ 0,04). Nous avons donc ajouté le critère de « contenance ». Une variante qui gardait la *plus grande* boîte a été essayée puis abandonnée : une grosse boîte parasite couvrant plusieurs panneaux les faisait tous disparaître. On garde donc la boîte *la plus confiante*.

5.6 Random Forest contre SVM

Nous avons comparé, sur les mêmes 25 images, le Random Forest et un **SVM linéaire** (descripteurs mis à l'échelle et scores ramenés entre 0 et 1 pour une comparaison juste). Le Random Forest gagne largement (tableau 4, figure 5) : le SVM linéaire sépare moins bien nos descripteurs, là où la forêt combine mieux les indices de forme, de couleur et de texture. **Nous gardons le Random Forest.**

TABLE 4 – Random Forest contre SVM linéaire (mêmes 25 images).

Modèle	AP	F_1	Rappel (détection)
Random Forest	0,284	0,429	—
SVM linéaire	0,069	0,194	0,149

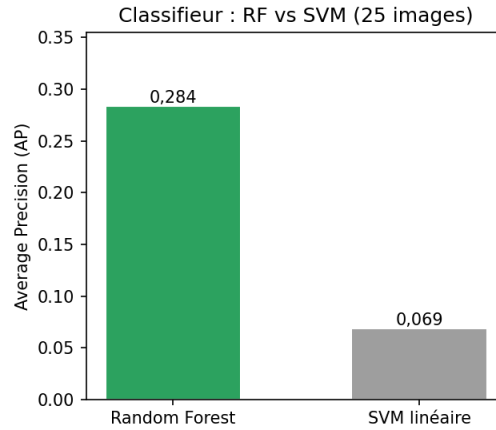


FIGURE 5 – Random Forest contre SVM linéaire (mêmes 25 images).

5.7 Préparer le fisheye par augmentation (essai non retenu)

Comme on ne peut pas savoir quelles images de test seront fisheye, nous avons préféré *apprendre* des panneaux déformés plutôt que de corriger les images au test. Nous avons donc ajouté une version déformée (effet « grand-angle ») de chaque panneau positif. Faute d’images fisheye en validation, le garde-fou était : « est-ce que ça casse le cas normal ? ». Réponse : oui, l’AP tombe de 0,284 à 0,221 sur 25 images, et réduire la déformation n’y change rien. La raison : on double le nombre de positifs, ce qui déséquilibre les exemples. Nous avons donc **désactivé** cette augmentation (le code reste disponible).

5.8 S’attaquer aux panneaux triangulaires (essai non concluant)

Sur les triangles, le modèle donnait parfois plus de confiance à une petite boîte posée sur un *coin* (deux arêtes nettes) qu’à la boîte cadrant tout le panneau. Nous avons tenté de récupérer ces coins comme contre-exemples. Mal réglé, cela étiquetait aussi des boîtes presque correctes en « pas panneau », et la détection s’effondrait. Même après réglage, le résultat n’était pas convaincant : approche **abandonnée** pour l’instant, mais conservée et documentée car elle éclaire le vrai problème (section 7).

6 Soumission sur le serveur d’évaluation

La détection sur les vraies images de test produit le fichier `detection.csv` soumis sur la page web de l’UTC. Notre première soumission (seuil de score 0,3) donne un **rappel correct** ($\approx 31\%$) mais une **précision minuscule** ($\approx 0,8\%$), d’où un F1 et une AUC plombés.

Diagnostic. Une précision de 0,8% veut dire que, pour 100 boîtes écrites dans le fichier, environ 99 sont fausses. Le coupable n’est pas le modèle en lui-même mais le *seuil de score* : nous l’avons laissé bas (0,3) — utile pour tracer la courbe AP, mais désastreux pour une soumission, car il laisse passer des milliers de fenêtres faiblement notées. C’est la version « image entière » du même piège qu’au tout début : un classifieur correct, noyé sous les fausses alarmes dès qu’on garde tout.

Correction. Pour la soumission, on relève le seuil de score à **0,6** : on ne garde que les détections sûres. L’effet, mesuré par une seconde soumission, est spectaculaire (figure 6 et tableau 5) : la précision est multipliée par 18 ($0,8 \rightarrow 15,2\%$), le rappel ne perd presque rien ($31 \rightarrow 26,9\%$), et le F1 comme l’AUC progressent fortement ($F1\ 1,6 \rightarrow 19,4\%$, $AUC\ 18,6 \rightarrow 23,7\%$). Le fichier passe de 385 ko à 18 ko. C’est un simple réglage de *point de fonctionnement*, sans réentraînement : le défaut dominant était bien le seuil, pas le modèle.

TABLE 5 – Effet du seuil de score sur la soumission UTC (scores du site, en %).

Soumission	Précision	Rappel	F_1	AUC
Seuil 0,3 (avant)	0,82	30,97	1,59	18,59
Seuil 0,6 (après)	15,16	26,87	19,38	23,68

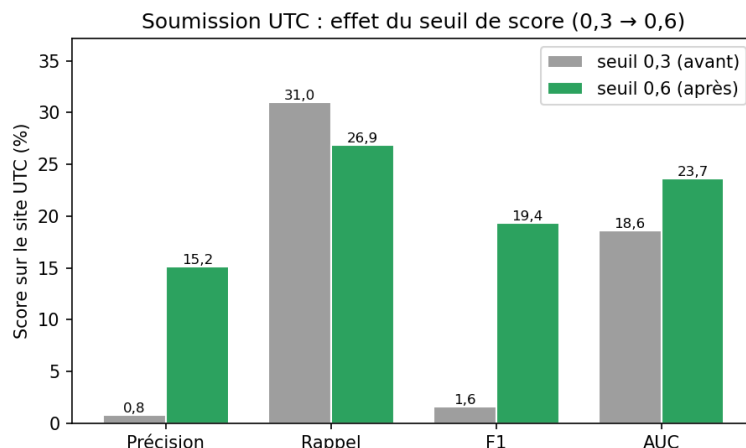


FIGURE 6 – Soumissions UTC avant/après le relèvement du seuil de score. À seuil bas la précision s’effondre ; à 0,6 elle remonte fortement sans presque perdre de rappel.

7 Analyse : quand ça marche, quand ça coince

Ce qui marche bien. Les panneaux nets, de taille moyenne à grande et bien colorés (rouge ou bleu sur fond clair) sont bien détectés et bien notés.

Ce qui coince.

- *Excès de fausses détections* à bas seuil : c’est le défaut dominant (section 6), maîtrisé en relevant le seuil de soumission.
- *Panneaux triangulaires* : le modèle peut préférer un coin du triangle au panneau entier. Le souci n’est pas la NMS mais le classifieur, qui note la présence de « texture de panneau » sans tenir compte du bon cadrage.
- *Boîtes avec trop de fond* : certaines détections contiennent plus d’arrière-plan que de panneau, signe que le modèle s’appuie trop sur le décor.
- *Petits panneaux lointains* : plus difficiles à trouver et à cadrer, à cause de la taille minimale de fenêtre et du pas.
- *Fisheye* : pas mesurable chez nous (aucune image fisheye en validation), donc robustesse incertaine.

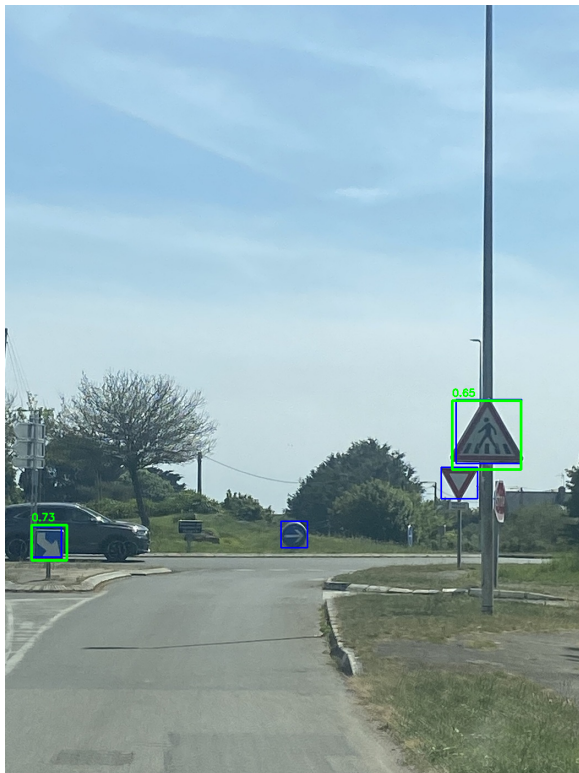
Illustration qualitative. Le dossier `images_results/` contient les images de test annotées (boîtes + scores) produites à chaque lancement. La figure 7 en montre un cas réussi et un cas d’échec typique.

Temps de calcul. L’étape lente est la fenêtre glissante : environ 130 s par image avec un pas de 5 px, ramené à ≈ 13 s avec un pas de 16 px, la prédiction répartie sur tous les cœurs et le pré-filtre couleur. La NMS ralentit aussi quand les détections sont très nombreuses (complexité en $O(n^2)$).

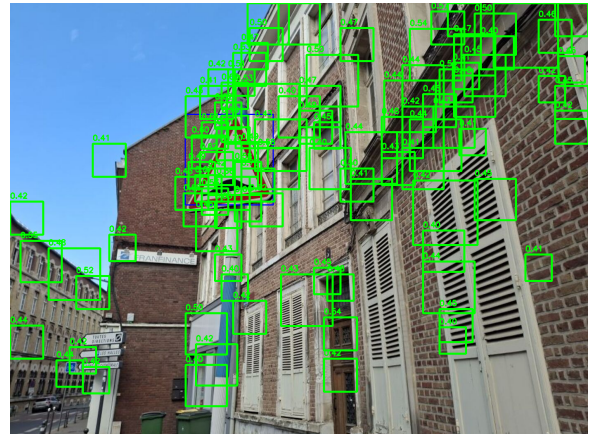
8 Conclusion et perspectives

En suivant notre fil conducteur — d’abord mesurer, ensuite améliorer — nous avons fait passer l’AP de 0,095 à 0,293 sur la validation, principalement grâce à un meilleur découpage des exemples et au hard negative mining ; la couleur et la texture apportent un gain supplémentaire. Tout aussi instructif : nous avons *écarté* sur preuve plusieurs idées (augmentation fisheye trop forte, hard negatives ciblés mal réglés, SVM linéaire), ce qui montre l’intérêt d’une évaluation rigoureuse. La première soumission a enfin révélé un défaut de précision lié au seuil de score : en le relevant à 0,6, l’AUC du serveur passe de 18,6 à 23,7 % et le F1 de 1,6 à 19,4 %.

La prochaine priorité est d’aider le classifieur à mieux « faire confiance » aux boîtes bien cadrées plutôt qu’aux sous-régions ou aux boîtes pleines de fond : enrichir les exemples positifs, plafonner le nombre de détections gardées par image, et calibrer les scores. Ces pistes devraient notamment aider sur les panneaux triangulaires et faire encore monter la précision en soumission.



(a) Réussite : panneau bien cadré et bien noté.



(b) Échec : sur-détection / boîte mal cadrée.

FIGURE 7 – Exemples qualitatifs (à choisir dans `images_results/` et copier dans `figures/`).

Déclaration d'utilisation d'IA générative

Conformément à l'énoncé, nous déclarons avoir utilisé un assistant d'IA générative pour : la mise en place des métriques et du protocole d'évaluation, l'aide au débogage, la génération des figures, et la rédaction de ce rapport. Exemples de requêtes employées :

- « Implémente les métriques de détection (IoU, précision/rappel, AP, F1) en code simple et commenté. »
- « Pourquoi mon AP est-elle faible alors que le score sur imageries est de 99 % ? »
- « Rends le hard negative mining multi-échelle. »
- « Pourquoi ma précision est-elle de 0,8 % en soumission, et comment la corriger ? »

Toutes les décisions, mesures et validations ont été contrôlées par les auteurs.

Contributions

Projet réalisé en binôme dans le cadre de l'UE SY32 (P2026) à l'UTC. Conception, implémentation, évaluation et rédaction ont été menées conjointement.